🦸 Fridge Hero — Technical Overview for HR / Interview

> A full-stack-in-one-file AI-powered recipe generation web application.

📌 Project Summary

Fridge Hero is a client-side web application built as a **single HTML file** (~60KB) that uses AI to generate recipes from user-provided ingredients. It works both online (via Google Gemini AI API) and completely offline (via a built-in JavaScript fallback engine).

🛠️ Tech Stack

Layer	Technology
Structure	HTML5 (semantic elements, datalist, form controls)
Styling	Pure CSS3 (custom properties, flexbox, CSS grid, keyframe animations)
Logic	Vanilla JavaScript (ES6+, async/await, Fetch API)
AI Integration	Google Gemini REST API (v1beta)
Storage	Browser `localStorage` API
Server	PowerShell `HttpListener` (local dev server)
No frameworks	Zero dependencies — no React, no jQuery, no npm

📐 Architecture

```
Single File App (index.html)
├── <style>          → All CSS (~835 lines)
│   ├── CSS Custom Properties (theme tokens)
│   ├── Layout: CSS Grid + Flexbox
│   ├── Glassmorphism card styles
│   ├── Keyframe animations (pan toss, particles, ripple)
│   └── Responsive breakpoints (mobile-first)
│
├── <body>           → Semantic HTML (~340 lines)
│   ├── Sidebar (Recipe Book history)
│   ├── Main Form (ingredients, preferences)
│   ├── Loading Card (frying pan animation)
│   └── Recipe Output Card
│
└── <script>         → All JavaScript (~430 lines)
    ├── State Management (in-memory + localStorage sync)
    ├── Gemini API calls with waterfall fallback
    ├── Offline Recipe Engine (regex + templates)
    ├── DOM manipulation & event handlers
    └── Visual effects engine (particles, ripple, flash)
```

🔑 Key Technical Concepts Demonstrated

1. API Integration with Waterfall Fallback

```
User clicks Cook →
  Try: gemini-2.5-flash
    → 503/429 error? Try: gemini-2.5-pro
      → error? Try: gemini-1.5-flash
        → error? → Run offline engine locally
```

- Uses native **Fetch API** with `async/await`
- Handles CORS errors, rate limits (429), server errors (503)
- Graceful degradation — app **never fails** to produce output

2. Offline-First Fallback Engine

- Pure JavaScript regex-based ingredient parser
- Structured template recipes for common ingredient combos
- No internet, no API key required in Demo Mode
- Detects ingredient categories (proteins, vegetables, carbs) and generates appropriate recipes

3. CSS Custom Properties (Theme System)

```
css
:root {
  --primary: #ff7a30;
  --bg: #f0f4ff;
  --glass-bg: rgba(255,255,255,0.55);
}
[data-theme="dark"] {
  --bg: #0f1117;
  --glass-bg: rgba(30,34,50,0.65);
}
```

- One toggle attribute swap changes the entire app theme
- Zero JavaScript re-renders needed for theming

4. Glassmorphism UI Pattern

```
css
backdrop-filter: blur(18px);
background: var(--glass-bg);
border: 1px solid var(--glass-border);
```

- Modern design trend using CSS `backdrop-filter`
- Frosted glass effect on all cards and panels

5. Keyframe Animation System

Three complex animation chains:

- **Frying Pan Loading Screen** — pan rock (`panToss`) + food fly (`tossFly`) + steam rise (`wispRise`) + spark pop (`sparkPop`)
- **Button Press Effects** — ripple expand, screen flash overlay, emoji particle burst using CSS custom properties (`--dx` , `--dy` , `--rot`) for randomized directions

- **UI micro-interactions** — fade-ins, slide-ups, pulse effects

6. LocalStorage Persistence

```
javascript
const LS = {
  THEME:   'fh_theme',
  KEY:     'fh_api_key',
  DEMO:    'fh_demo',
  HISTORY: 'fh_history'
};
```

- Theme preference survives page refresh
- Full recipe history persists across sessions
- Cross-session API key storage

7. Dynamic DOM Manipulation

- Ingredient rows added/removed at runtime without any framework
- HTML5 `<datalist>` for 40-item autocomplete suggestions
- Recipe history sidebar dynamically rendered from `localStorage`

📐 Design Patterns Used

Pattern	Where Used
Module Pattern	JS wrapped in self-contained functions
Observer Pattern	`change` event listeners on form inputs
Strategy Pattern	API model waterfall fallback chain
Template Method	`buildOfflineRecipe()` selects recipe template by ingredient type
Facade Pattern	`handleCook()` orchestrates all sub-systems (validation → API → parse → display → save)

🧪 Error Handling Strategy

```
Input Validation → No ingredients? → Show inline error, stop
API Error Types:
  TypeError (network)  → Show CORS warning + run offline engine
  HTTP 429/503         → Retry with next model in chain
  Invalid JSON         → Fallback to regex text parser
  All models fail      → Always show offline recipe (never crash)
```

💾 Data Flow

```
User Input (ingredients, prefs)
  ↓
handleCook() validates input
  ↓
fireCookEffects() — visual burst on button
  ↓
setLoading(true) — show frying pan animation
  ↓
callGeminiAPI() OR buildOfflineRecipe()
  ↓
parseRecipeResponse() — regex extracts name, steps
  ↓
displayRecipe() — renders card with banner image
  ↓
saveToHistory() — persists to localStorage
  ↓
setLoading(false) — show recipe card
```

🚀 Performance Considerations

- **Zero network dependencies** for core functionality (offline mode)
- **Non-blocking font loading** — Google Fonts loaded with `media="print"` trick; system fonts used as fallback
- **DOM cleanup** — particle elements self-remove after animation via `setTimeout`
- **Debounced loading state** — minimum 4.2s display for smooth UX (prevents flash of loading screen)

- **No framework overhead** — pure vanilla JS keeps bundle size at ~60KB total

🌐 Local Development Setup

```
powershell
# Option 1: Double-click
run_server.bat

# Option 2: PowerShell
.\serve.ps1

# Then open:
http://localhost:8000
```

> **Why a local server?** Google Gemini API blocks requests from `file://` URLs due to CORS policy. The PowerShell `HttpListener` serves the file over `http://` to bypass this.

📊 Project Stats

Metric	Value
Total file size	~60 KB
Lines of code	~1,612
CSS lines	~835
JavaScript lines	~430
External dependencies	**0**
npm packages	**0**
Frameworks	**None**
Works offline	✅ Yes

💡 What This Project Demonstrates

- ✅ Strong **HTML/CSS/JavaScript** fundamentals without framework crutches
- ✅ **REST API integration** with production-grade error handling
- ✅ **Offline-first** thinking and graceful degradation
- ✅ **Modern CSS** — custom properties, glassmorphism, complex keyframe animations
- ✅ **UX thinking** — loading states, visual feedback, history persistence
- ✅ **Clean architecture** in a constrained single-file environment